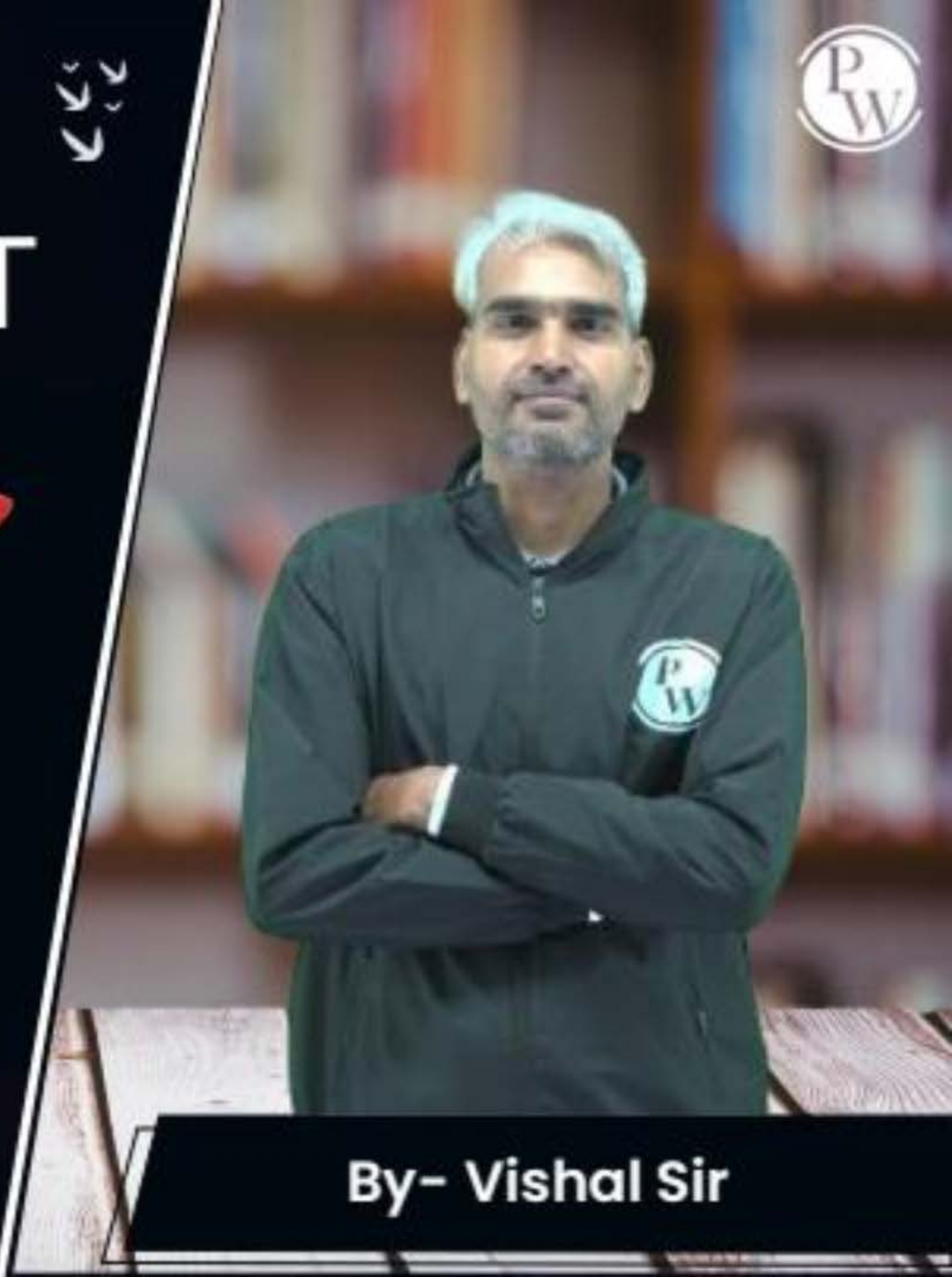
Computer Science & IT



Database Management System

File organization & Indexing

Lecture No. 05



By- Vishal Sir

Recap of Previous Lecture



Topic Structure of nodes of B Tree Index



Slide

Topics to be Covered

Topic	B Tree Index



Slide



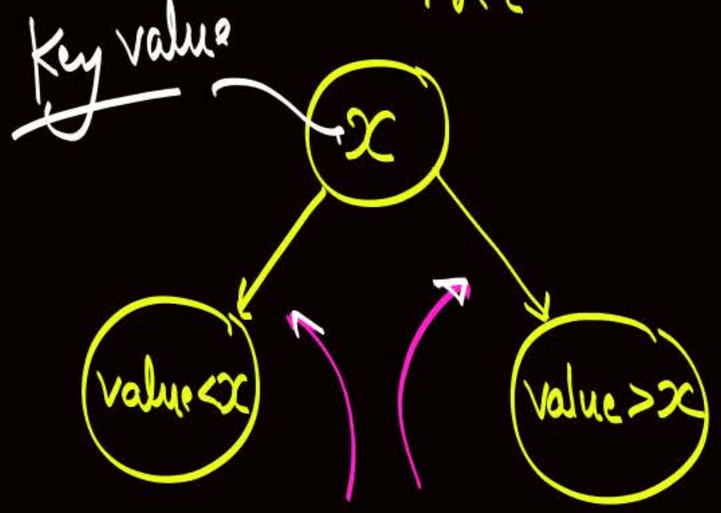
Topic : Dynamic Multi-level Index

Multi-way Search Tree → { B Tree, B+ Tree } } Balanced Search Tree

Balanced Search Tree ↓

All leaf nodes of the tree will be at same level.

Binary Search Tree

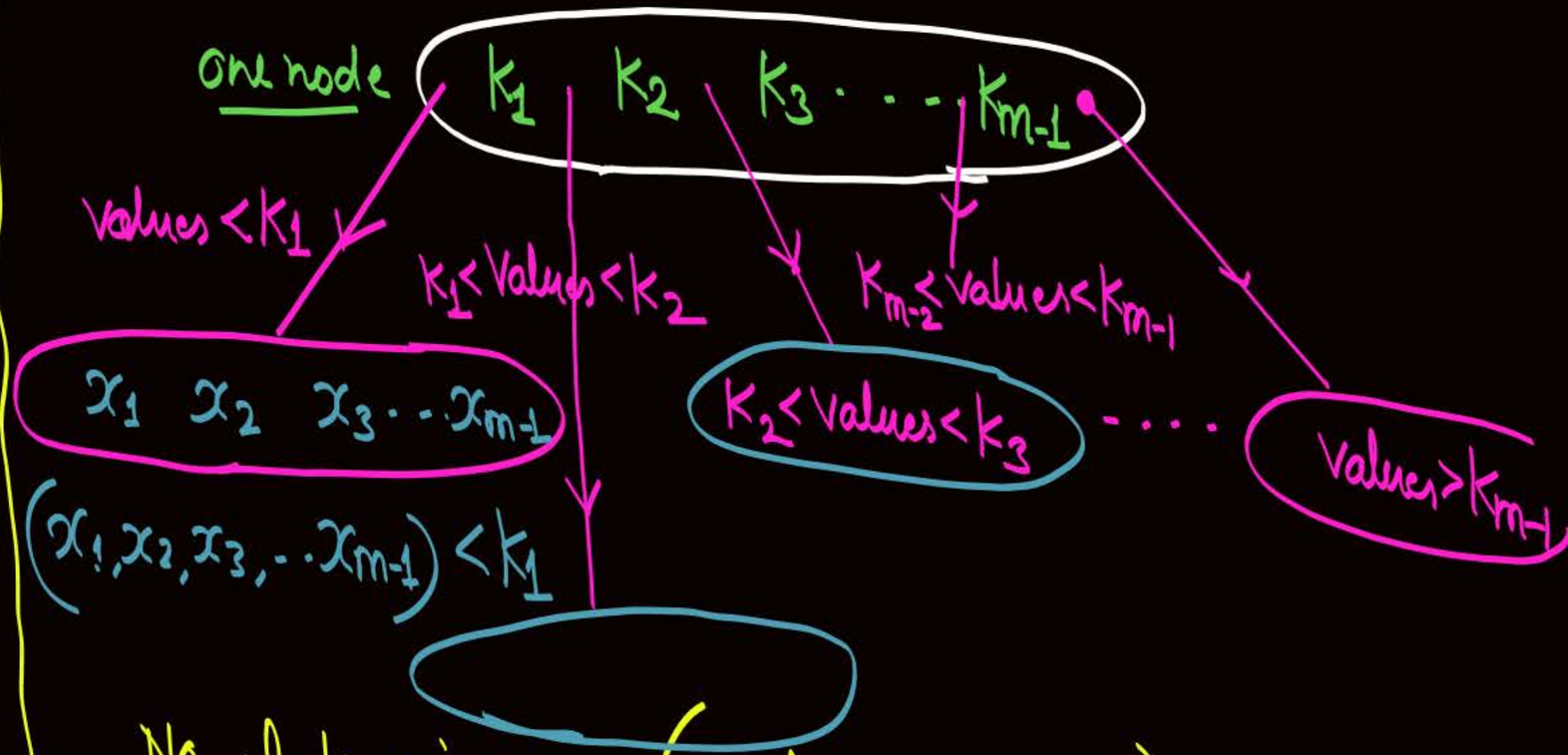


Maximum two children of a node in B.S.T.

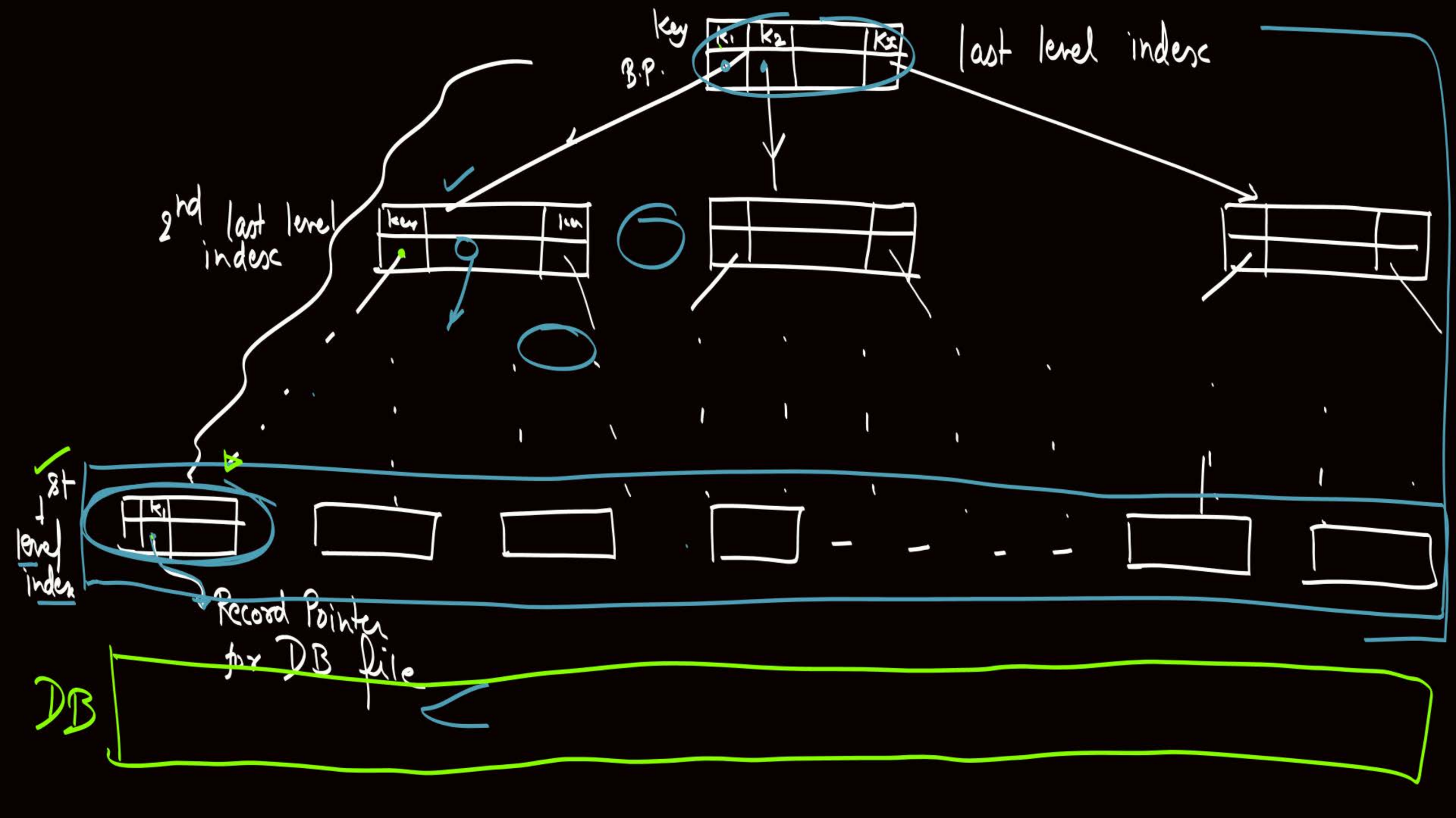
∴ It is a 2-way Search tree.

M-Way Search tree

i.e. Maximum 'm' children of a node



No. of keys in a node = (No. of Child Pointer − 1)


Key
B.P.
K1
K2
Kn
last level index
2nd last level index
1st level index
Record Pointer for DB file
DB



Topic : B Tree

* Order of a node of B Tree: Order of a node of B Tree index is defined as maximum number of Child pointer a B Tree node can contain

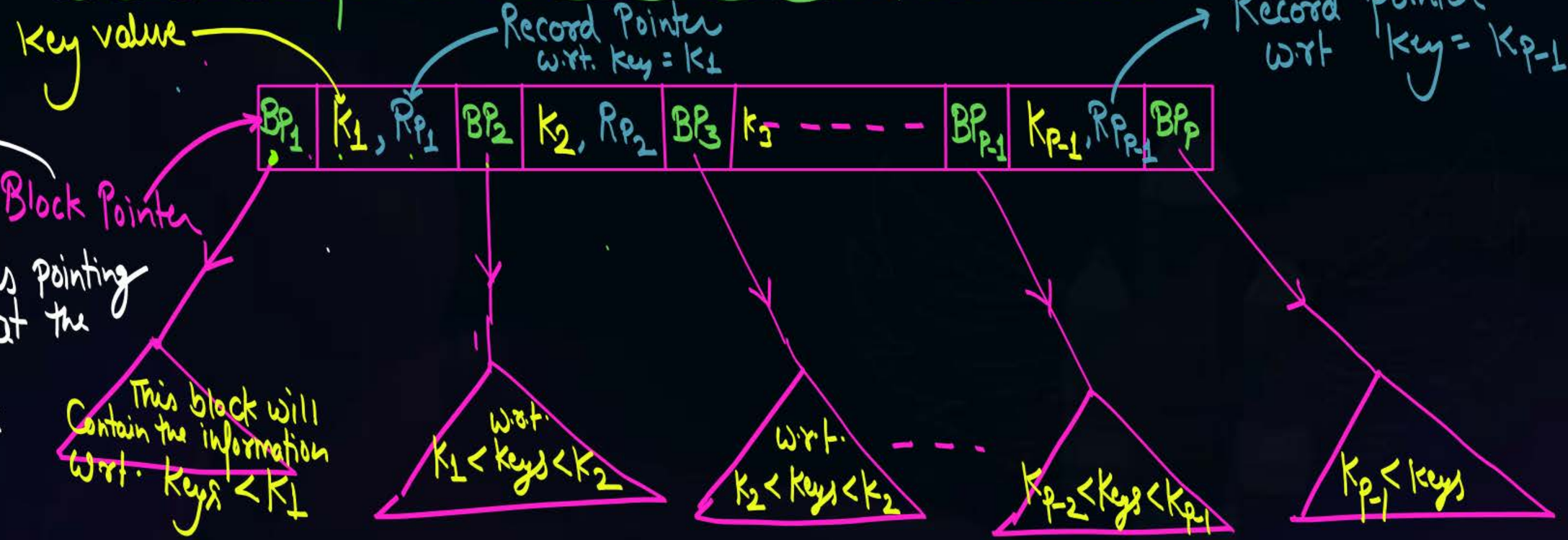


Topic : B Tree

{ i.e. Maximum number of child pointer possible = P
∴ Maximum number of keys possible in a node of B Tree = (P-1)

Let order = P for a node of B Tree index

(1) Structure of an internal node of B Tree :-



Note: In B Tree index, the record pointers corresponding to the keys may be present in the internal node of B Tree as well.

This will not happen
w.r.t. Static multi-level index
(or)
w.r.t. B+ Tree.

→ In these two cases record pointers will be present only at the last level of Index tree.

* In each node of B Tree along with key values their corresponding record pointer will also be available



Topic : B Tree

* Order = Max. no. of child pointer possible

* Record Pointer (Rp) : Record pointer is the pointer pointing to the database record corresponding to the key value

* Block pointer (Bp) / Tree pointer / Child pointer / Node Pointer :-

  → It is the pointer pointing to the blocks of the index tree at the next level.



Topic : B Tree

{ i.e. Maximum number of child pointer possible = P
∴ Maximum number of keys possible in a node of B Tree = (P-1)

Let order = P for a node of B Tree index

(2) Structure of a leaf node of B Tree :-



Block Pointer

Block Pointers are the pointers pointing to the blocks at the next level in B Tree index

NULL NULL NULL NULL NULL



Topic : B Tree

* In a B Tree of order = P,

for both internal as well as leaf node as well as root node

(i) Maximum number of child pointer = P

(ii) Maximum number of keys possible in a B Tree node = (P-1)



Topic : B Tree

* In a B Tree of order = P,

(iii) { Each node { Except root node } will Contain at least $\lceil \frac{P}{2} \rceil$ block pointer and $\left(\lceil \frac{P}{2} \rceil - 1 \right)$ keys

(iv) Root node will Contain at least 2 block pointer and $(2-1) = 1$ key



Topic : B Tree

* In a B Tree of order = P,

(v) All the keys within a node of B Tree must be in ascending order.

(vi) All the leaf nodes must be at the same level



Topic : B Tree

where order defines the maximum number of Block Pointer a node can have

* In a B Tree of order = p

(i) Max No. of block pointer possible = p

(ii) Max No of keys possible in a node = p - 1

(iii) Min No. of block pointer a non-root node must have = $\lceil \frac{p}{2} \rceil$

(iv) Min No. of keys a non-root node must have = $\lceil \frac{p}{2} \rceil - 1$

(v) Min No. of block pointer a root node must have = 2

(vi) Min No. of keys a root node must have = 1



Topic : B Tree

* In a B Tree of order = P

Where Order is defined as maximum number of keys a B Tree node can have

(i) Max No. of block pointer possible = (P+1)

(ii) Max No of keys possible in a node : P

(iii) Min No. of block pointer a non-root node must have : $\left\lceil \frac{P+1}{2} \right\rceil$

(iv) Min No. of keys a non-root node must have : $\left\lceil \frac{P+1}{2} \right\rceil - 1$

(v) Min No. of block pointer a root node must have = 2

(vi) Min No. of keys a root node must have = 1

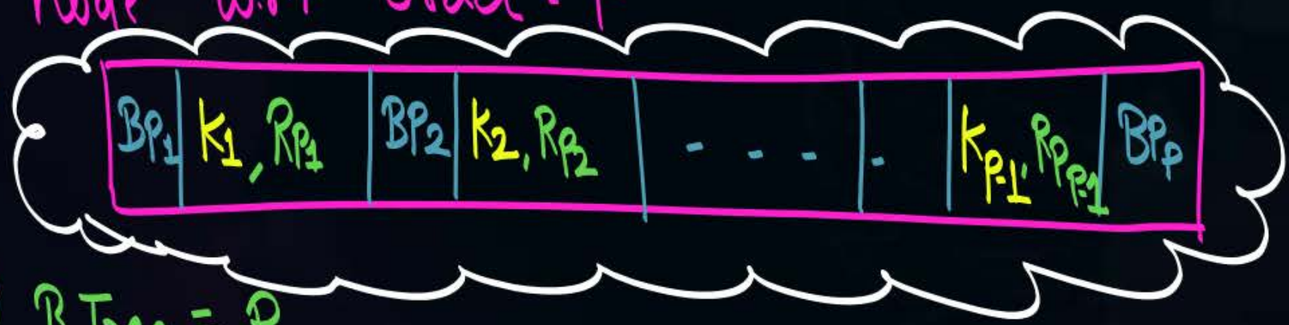


Topic : B Tree

Note: A node of B Tree will be a block of disk

∴ Whatever we want to store in a node of B Tree must fit into a block of disk

∴ B Tree node w.r.t order = P



← it must fit into a single block of disk

i.e. if order of B Tree = P

$$\left[(P * \text{Block pointer size}) + (P-1) * \{\text{Key value Size} + \text{Record pointer Size}\}\right] \leq \text{One disk Block size}$$



Topic : B Tree

Consider a relations R with a key filed K. A B-tree of order P is used to access structures on K. Where 'P' denotes the maximum Number of tree pointers in a B-tree index node. Assume that key (k) is 30 bytes long disk block size 1024 byte. Each data (Record) pointer size P_D is 12 bytes long & each block pointer P_B is 10 byte long. In order for each B-Tree Node to fit in a single disk block, the maximum value of P is-

$$(P * 10) + (P-1) * (30+12) \leq 1024$$

$$52P - 42 \leq 1024$$

$$P \leq \left(\frac{1024+42}{52}\right)$$

$$P \leq 20.5$$

A 19

B 20

C 21

D 22

Insertion in B tree :- { Insertion will always happen at leaf node }

1. Search for the leaf node which will hold the key value
2. If leaf node have space {i.e. does not contain (P-1) keys}, then simply insert the key in ascending order
3. If leaf node is full {i.e., already contain (P-1) keys} then insert the key in the ascending order then split that node into two parts, and promote the median position key to the parent node

3ⓐ:- If Parent node have space to accomodate a new key, then no further action is required

3ⓑ:- If Parent node is already full, then split the parent node in similar manner, this may go upto root node, if root node is also already full then split the root node as well, the key in the median position of root node will become the new root node of the tree. and height will increase by "1"

Q: let order p = 4 {Max. no. of Child pointers = 4}
and the keys to be inserted are
90, 10, 2, 8, 30, 94, 14, 28, 38, 20, & 25
in the same order.

* We know order = 4,
  * Max. no. of keys possible = (4-1) = 3
  * Min no. of keys required in non-root node : $\lceil \frac{4}{2} \rceil - 1 = 2 - 1 = 1$
  * Min no of keys required in root node = 1 {always}

90, 10, 2, 8, 30, 94, 14, 28, 38, 20, 25

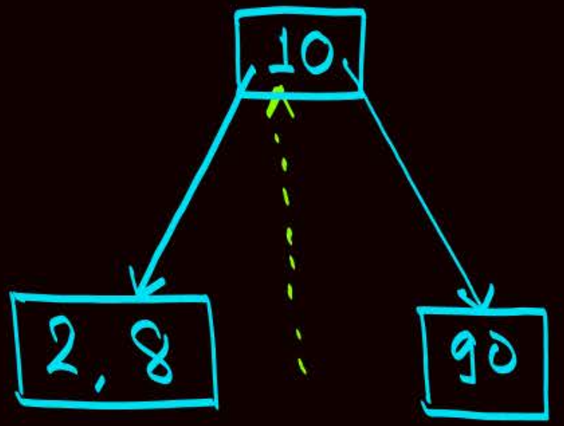
90 → [90]

10 → [10, 90]

2 → [2, 10, 90] {Full}

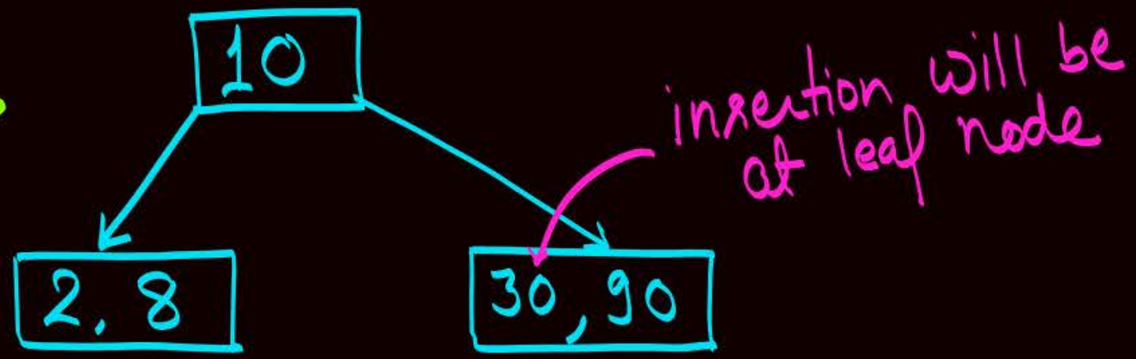
8 → [2, 8, 10, 90] {overflow} ∴ Split ⇒

Median

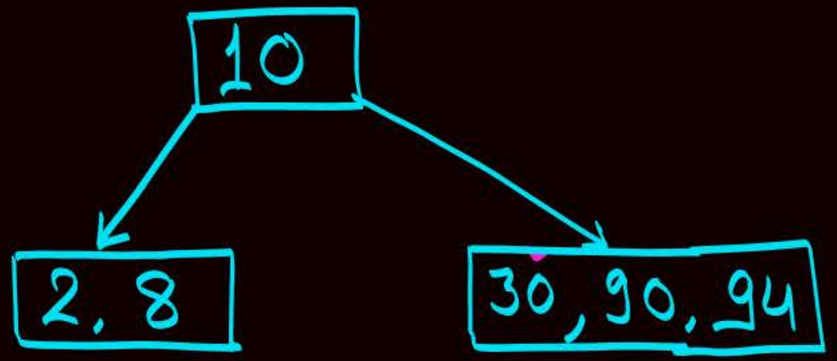


90, 10, 2, 8, 30, 94, 14, 28, 38, 20, 25

30 →

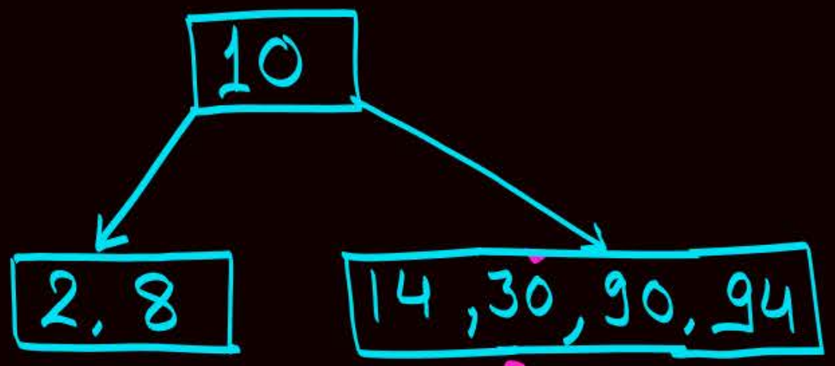


94 →

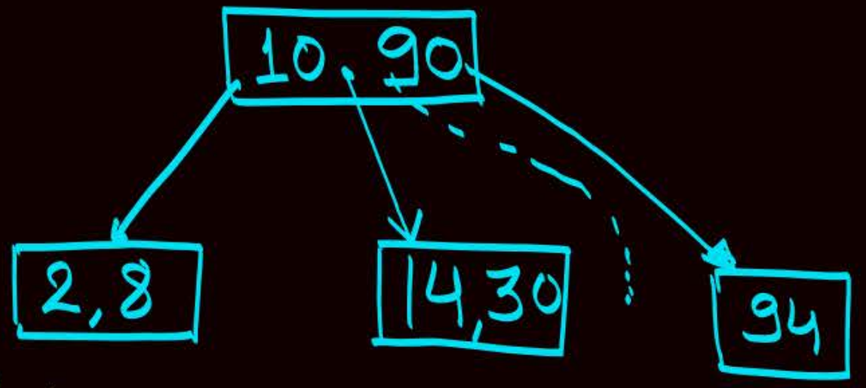


90, 10, 2, 8, 30, 94, 14, 28, 38, 20, 25

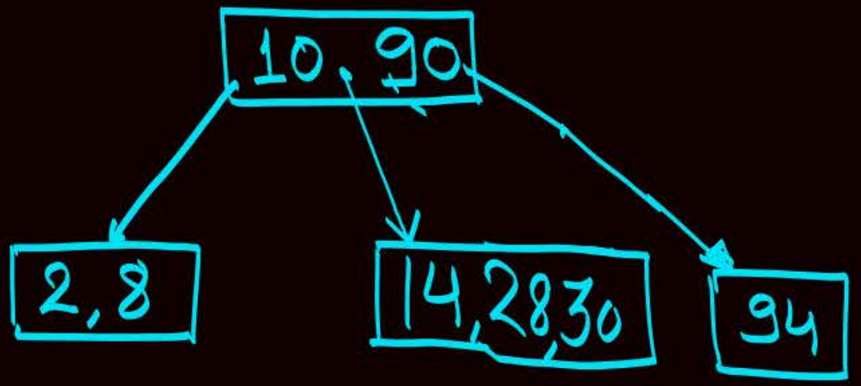
14 →



{overflow} ⇒ ∴ Split ⇒

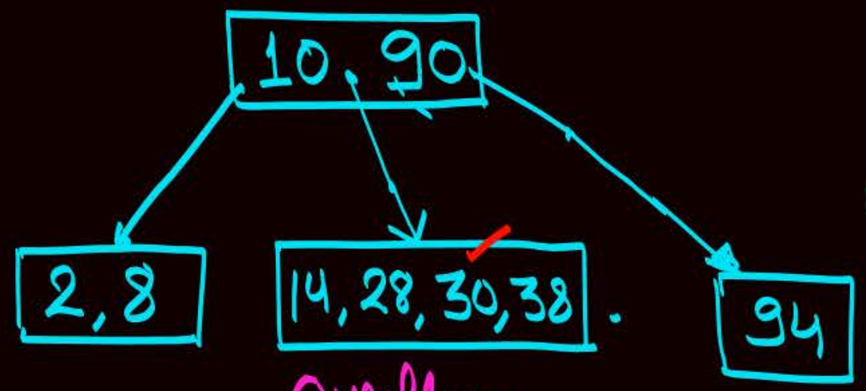


28 →



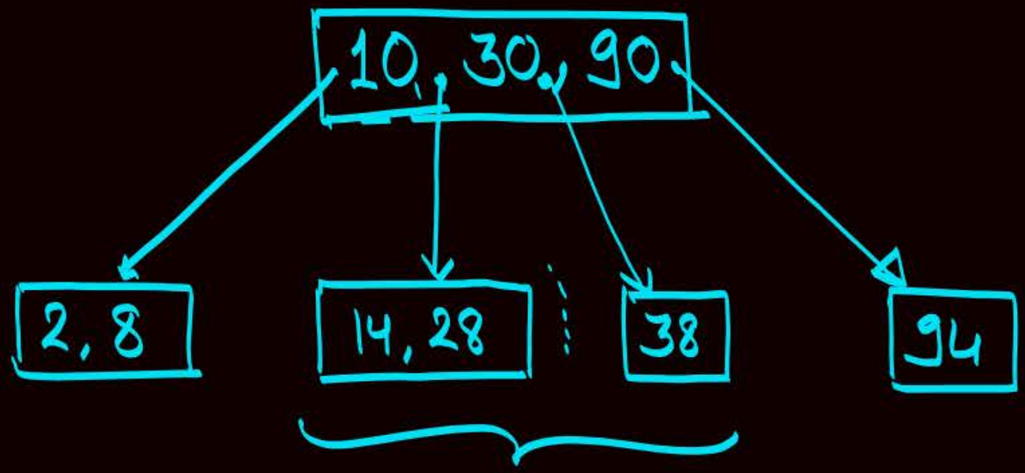
90, 10, 2, 8, 30, 94, 14, 28, 38, 20, 25

38 →

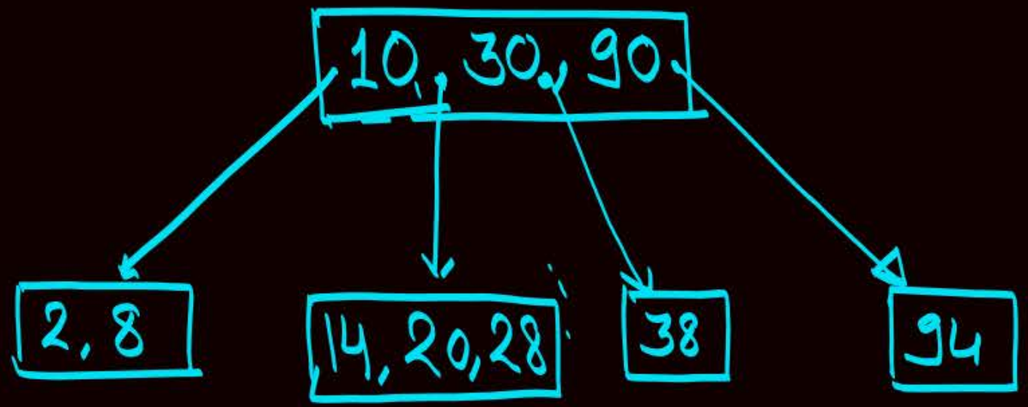


Overflow
{∴ Split}

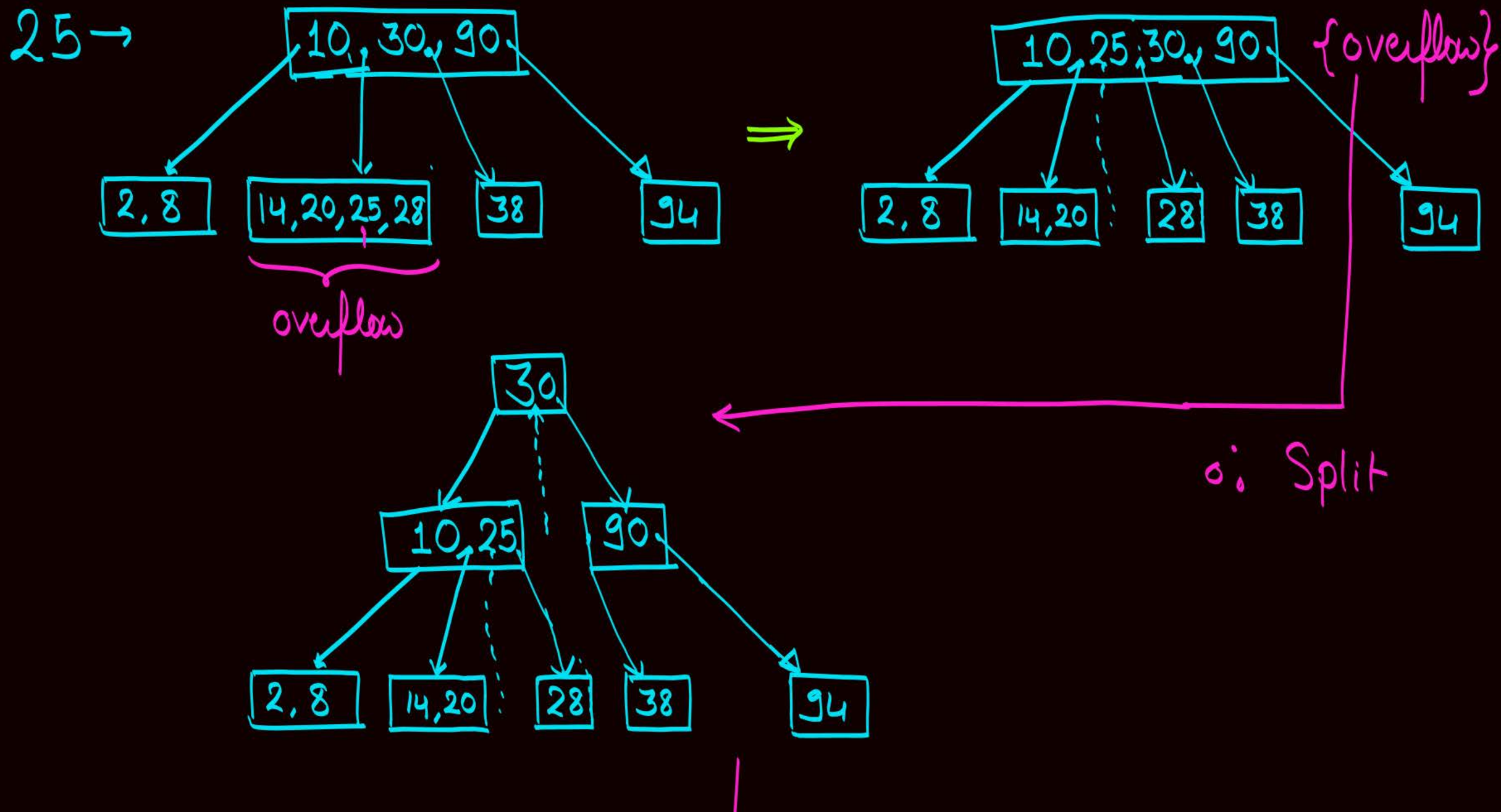
⇒



20 →



90, 10, 2, 8, 30, 94, 14, 28, 38, 20, 25



Deletion from a B Tree : { key to be deleted may be Present in a leaf node or in an internal node }

If key to be deleted is not in leaf node, then replace it with inorder predecessor {largest key in left sub-tree} or inorder successor {smallest key in Right subtree}, and then delete the key from leaf node using following approach,

Deletion from a B Tree :

(1) If the leaf node from which key will be deleted contains more than the minimum number of keys required in it $\{$ i.e $> \lceil \frac{p}{2} \rceil - 1 \}$, then simply delete the key from that leaf node. {No further action is required}

Deletion from a B Tree :

② If the leaf node from which key will be deleted contains exactly the minimum number of keys required in it {i.e $= \lceil \frac{p}{2} \rceil - 1$} then after deletion of the key this node will become deficient node {i.e. underflow}

Deletion from a B Tree :

2 (a) :- Check with left sibling & right sibling respectively if they can help. If any one of the sibling can help {i.e. contain more than minimum number of keys required} then borrow from that sibling {via parent}

Case (i): borrow from left sibling :- Largest key of the left sibling (Redistribute) will be propogated to parent node, and a key from the Parent node will be transferred to defficient node

Case (ii): Borrow from Right Sibling :- Smallest key of the right sibling will be (Redistribute) Propogated to the parent node, and a key from the parent node will be transferred to defficient node.

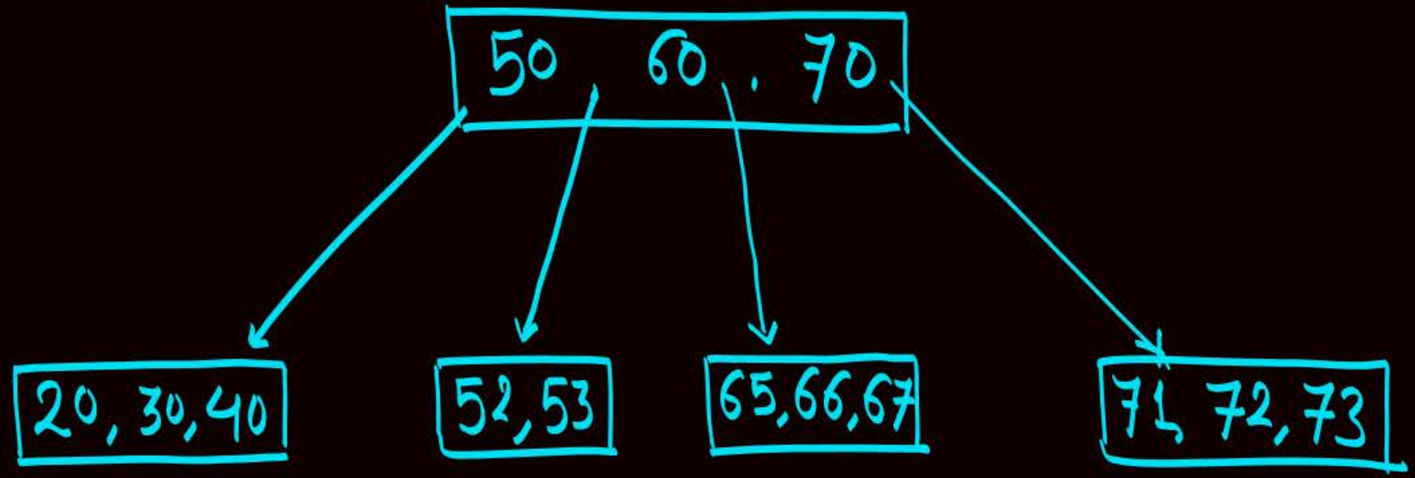
Deletion from a B Tree :

2 (b) :- If none of the immediate sibling (left/Right) can help, then merge the deficient node with one of its sibling and one key from the parent node.

As a result of this if parent node becomes deficient {i.e. Contains less than the minimum no of keys required}, then repeat the same w.r.t. parent node

It may continue upto root node, If root node also becomes deficient then height of the tree will reduce by "1".

Q:- Consider the following B tree :-



Let order of B tree = 5

Delete the following elements from the above B Tree

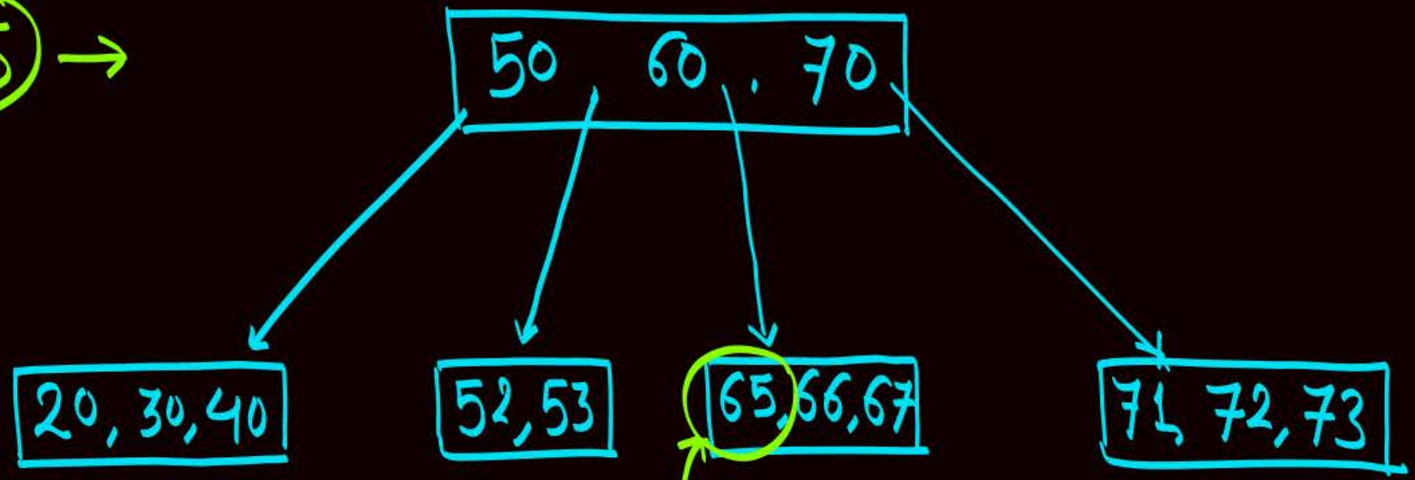
65, 53, 66, 52 & 60 in the same order

order (P) = 5

- Maximum No. of keys = 4
- Min No. of keys w.r.t non-root node : $\lceil \frac{5}{2} \rceil - 1 = 2$
- Min no. of keys w.r.t. root node : 1 {always}

Q:- Consider the following B tree :-

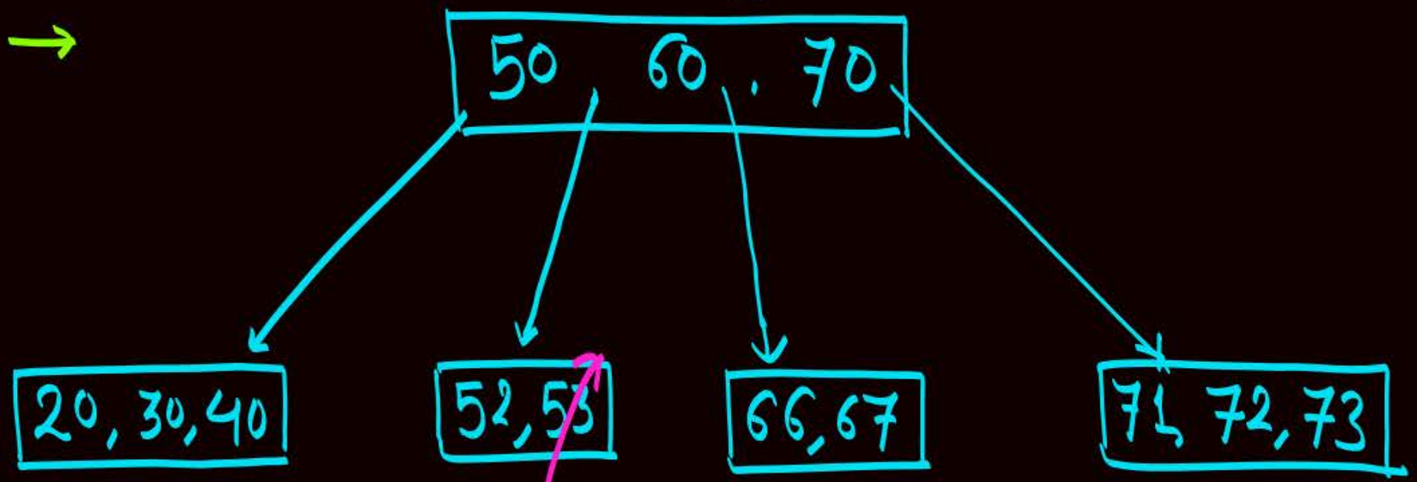
delete (65) →



already at the leaf node
(and)
no. of keys are 3 which is more than minimum required
∴ Simply delete

Q:- Consider the following B tree :-

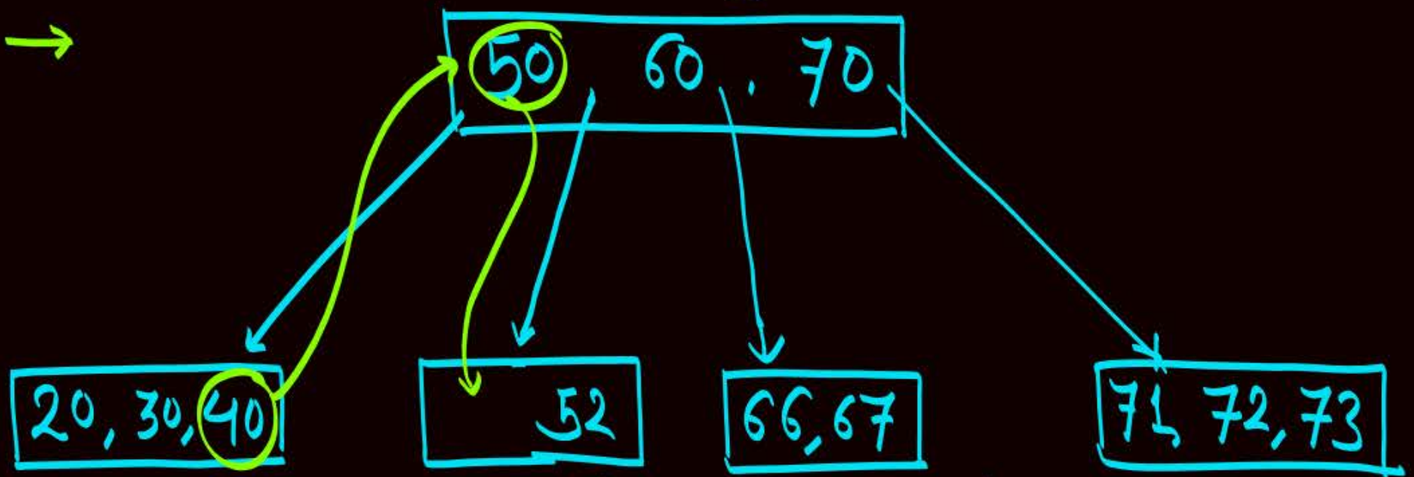
delete (53) →



already at leaf node ∴ delete it,

Q:- Consider the following B tree :-

delete (53) →



Underflow after deletion

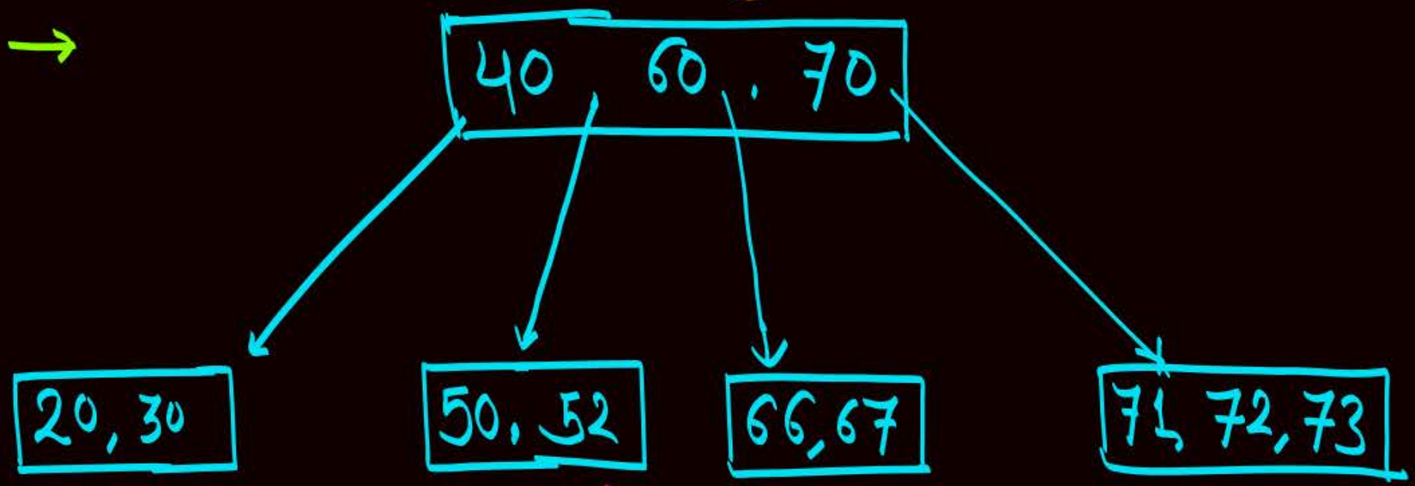
↓

left sibling contains more than the minimum no. of keys required

∴ Borrow from left sibling through parent node

Q:- Consider the following B tree :-

delete (53) →



Underflow

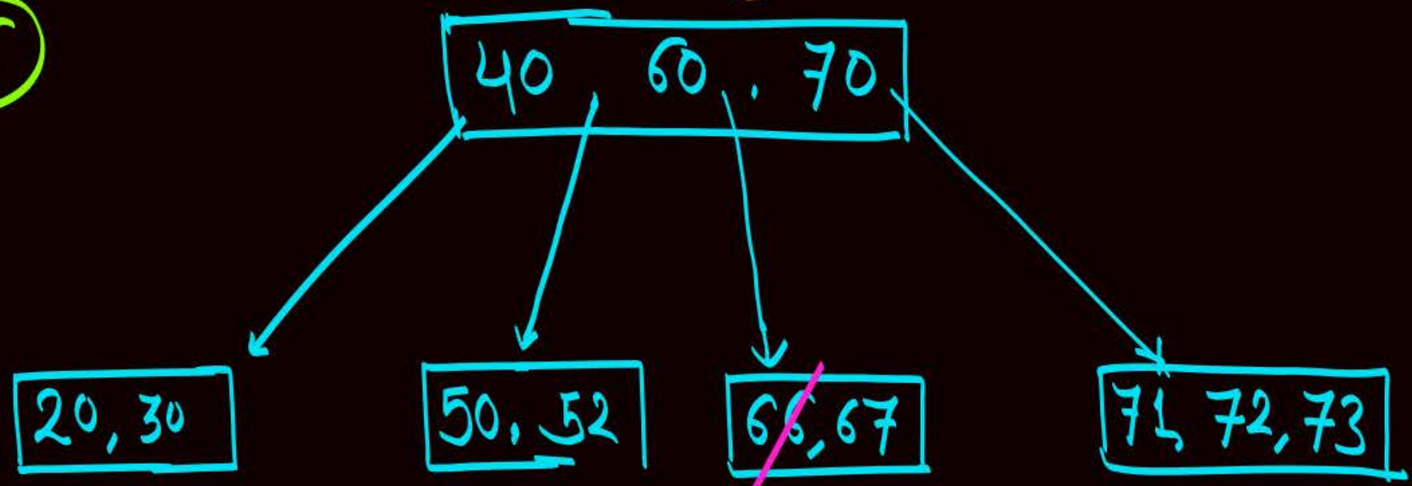
↓

left sibling contains more than the minimum no. of keys required

∴ Borrow from left sibling through parent node

Q:- Consider the following B tree :-

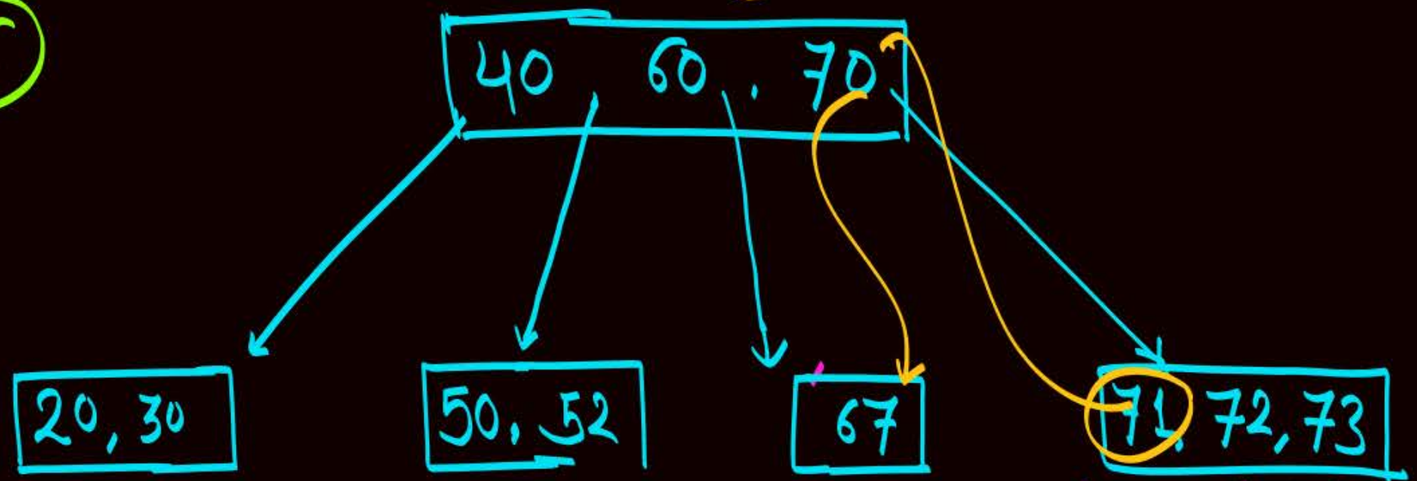
delete (66)



Already at the leaf node ∴ Simply delete it

Q:- Consider the following B tree :-

delete (66)



Underflow after deletion

{ Ask siblings }

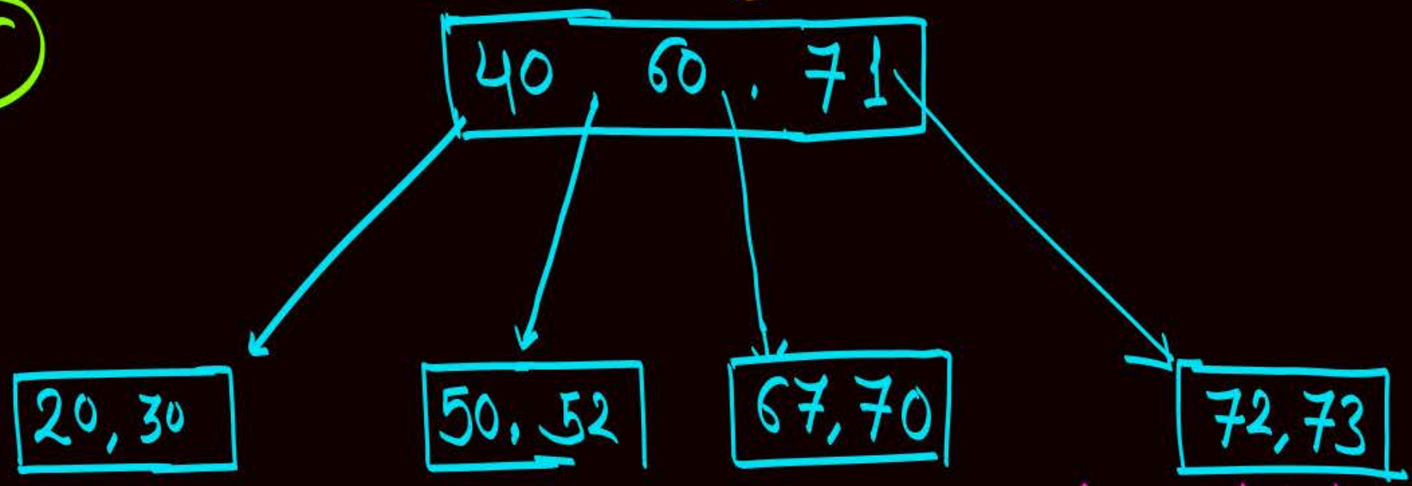
(i) Left sibling can not help

(ii) Right sibling can help

∴ Borrow from right sibling through parent.

Q:- Consider the following B tree :-

delete (66)



Underflow after deletion

{ Ask siblings }

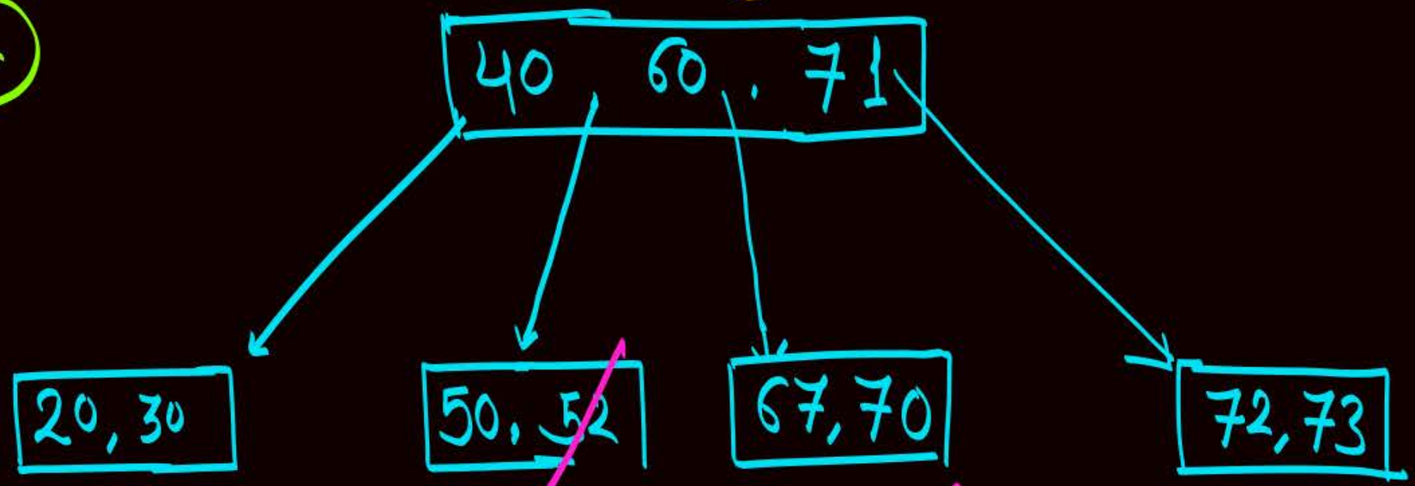
(i) Left sibling can not help

(ii) Right sibling can help

∴ Borrow from right sibling through parent.

Q:- Consider the following B tree :-

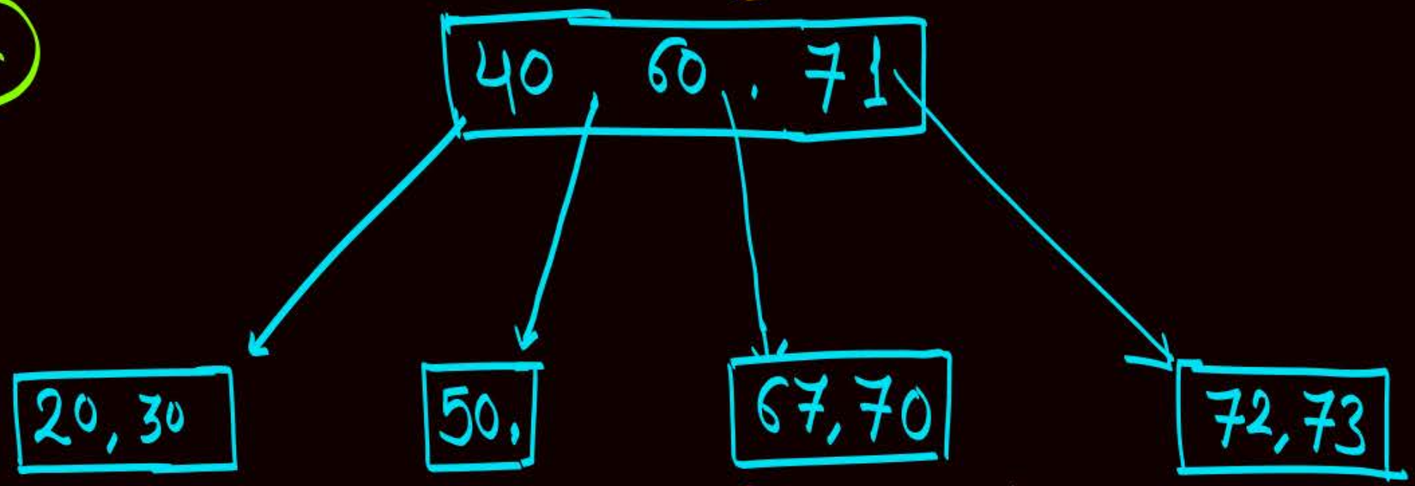
delete (52)



already at leaf node, so delete it

Q:- Consider the following B tree :-

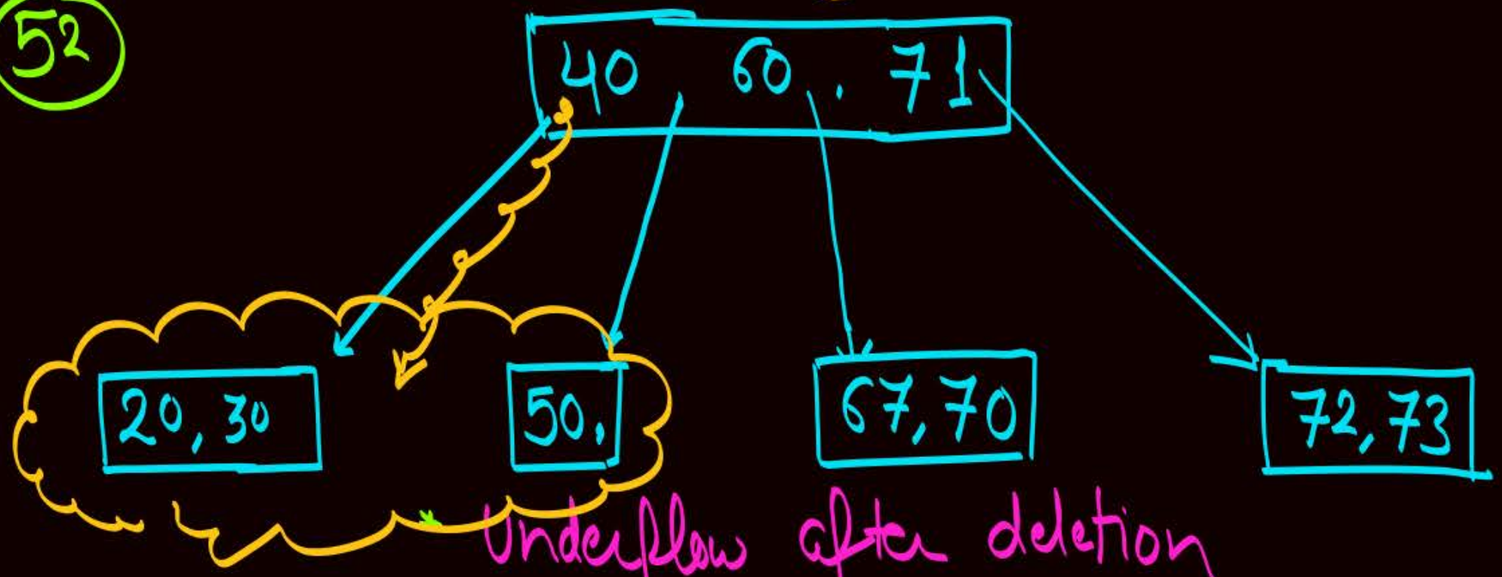
delete (52)



* Underflow after deletion
* None of the sibling can help; ∴ Merge with one of them via one key from parent

Q:- Consider the following B tree :-

delete (52)

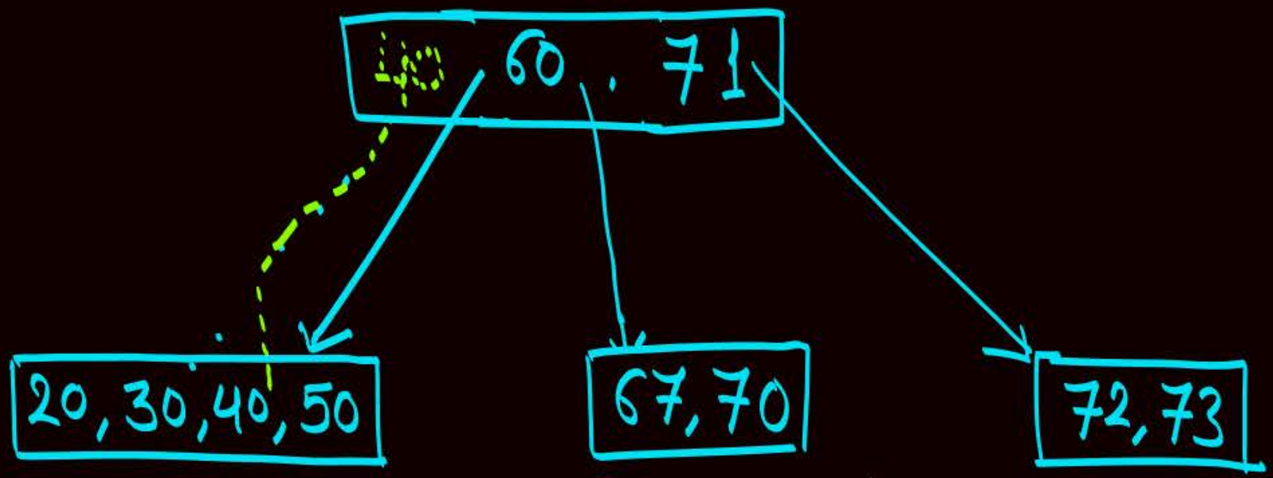


Underflow after deletion

- None of the sibling can help; ∴ Merge with one of them via one key from parent

Q:- Consider the following B tree :-

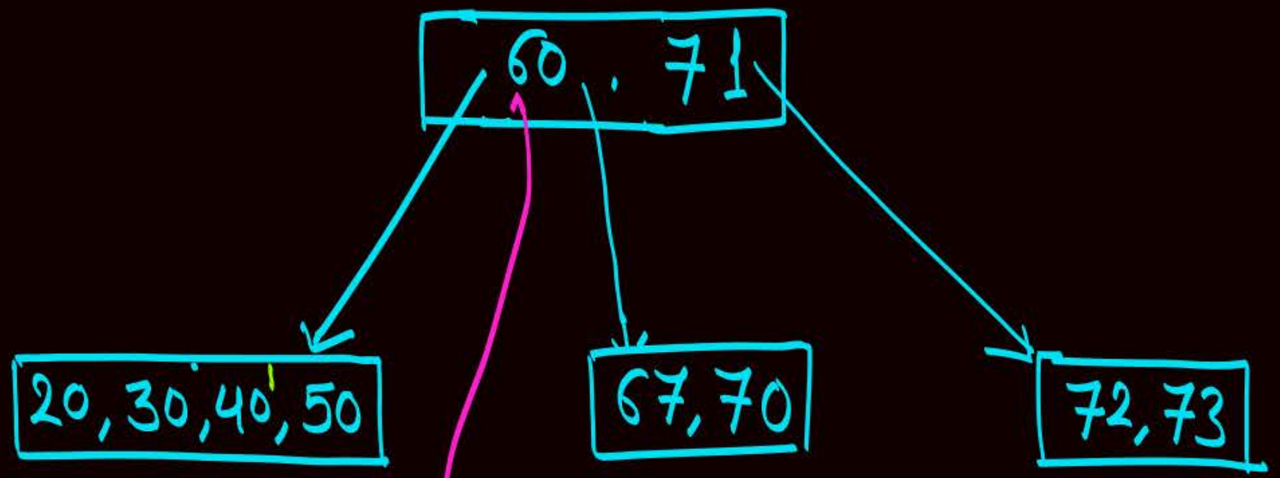
delete (52)



* Underflow after deletion
* None of the sibling can help, so Merge with one of them via one key from parent

Q:- Consider the following B tree :-

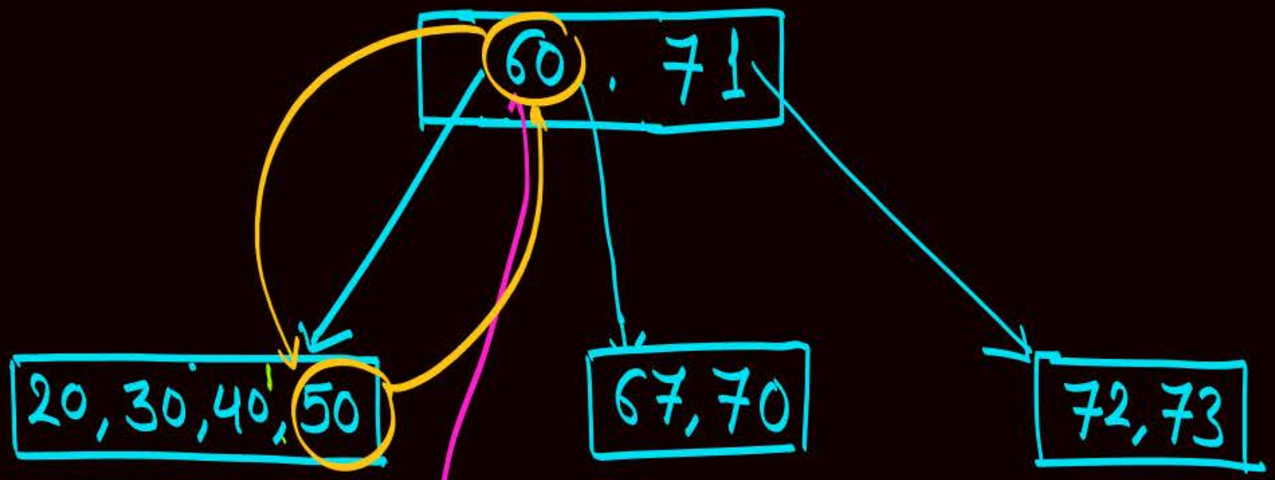
delete (60)



60 is in a non-leaf node

Q:- Consider the following B tree :-

delete (60)



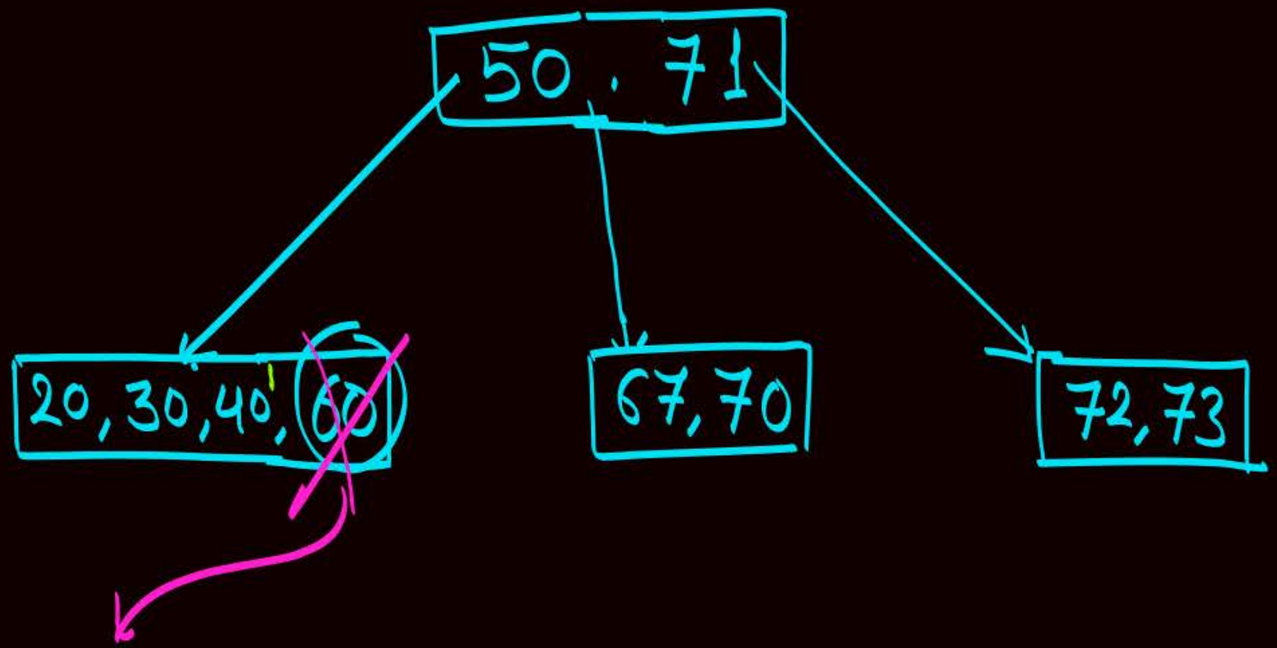
60 is in a non-leaf node

∴ Swap with inorder successor (or) predecessor

Here, Swap with inorder predecessor because corresponding node contains more no of keys

Q:- Consider the following B tree :-

delete (60)



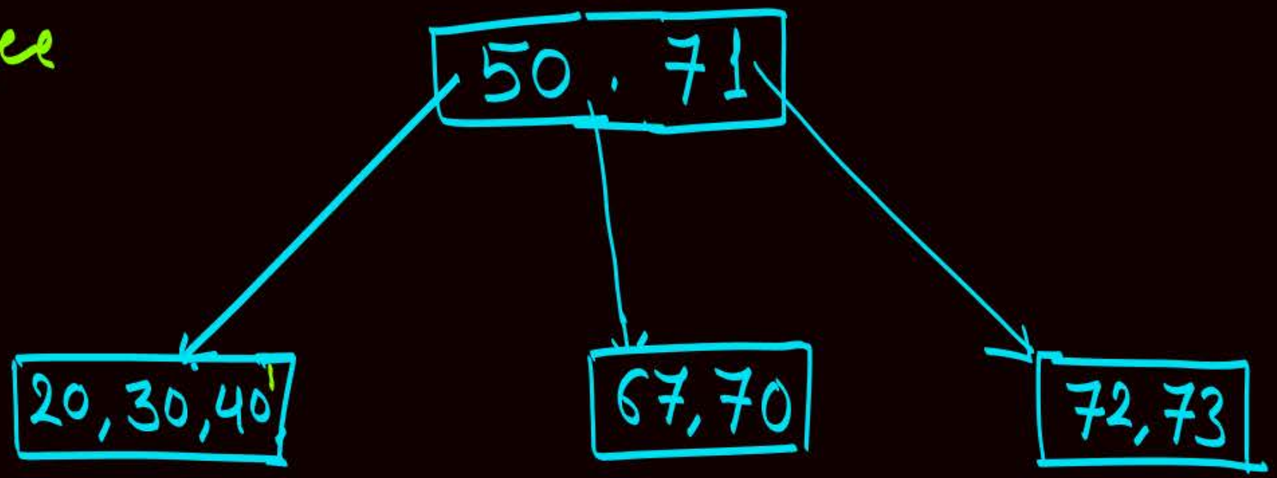
now delete it

∴ Swap with inorder successor (or) predecessor

Here, Swap with inorder predecessor because corresponding node contains more no of keys

Q:- Consider the following B tree :-

Final B Tree after all deletions



Q: Let order of B Tree is P: -

Identify the following

① Minimum number of keys (required) in a B Tree of order = P, and height = h

② Maximum number of keys possible in a B Tree of order = P, and height h.

③ Minimum number of Nodes in a B Tree of order: P & height = h

④ Maximum number of nodes in a B Tree of order = P & height = h

Root is at height = 0 & level = 1



2 mins Summary

Topic B Tree Index

Slide

THANK - YOU